SeatRelay: Unified Registration and Timed-Hold Seat Relay for Multi-Session Workshops

CS5224 Cloud Computing — Group 18 — Final Report

[PENDING: team member names and student numbers]

Public repository: https://github.com/gengyuzccc-cell/Team18_CS5224_SeatRelay_Project_AY2627Sem1

1 Motivation and business case

Problem. Many workshops on campus and in continuing education and training (CET) run the same content in several sessions so that people with different timetables can attend. Registration is usually one form per session plus a spreadsheet. A participant who could attend two sessions registers for both to be safe; the organiser later has to find and release the duplicate seat by hand, and a seat freed by a dropout stays empty unless someone e-mails the next person on a waiting list. Duplicate holds block seats that other people want, and the reconciliation is manual work for the organiser.

SeatRelay. A participant registers *once* per workshop with an ordered list of acceptable sessions and joins *one* queue; an account holds at most one seat per workshop. A free seat is offered to the first queued person who accepts that session, and the offer is held for a time limit (`holdSeconds`, default 900 s). A seat released by an expiry, a decline, a cancellation or a capacity increase is offered to the next person in the same decision (Fig. 1). The organiser manages 2–5 sessions and sees, per session, capacity, confirmed and held seats, the queue, a backfill log with latencies, distinct participants served, attendance and a CSV export. Participants see their status and queue position and receive in-app and e-mail notifications.

Evidence that repeated sessions occur. On the NUS Libraries calendar from 2026-08-01 to 2026-10-16, 11 of the 37 events tagged Workshop were repeats of 2 titles (a library tour 8 times, an orientation workshop 3 times) [1]; on the SMU Libraries calendar in the same window, 2 titles ran twice among 44 events [2]. These are our counts of the public calendars, captured and counted by a script (`results/market/`). Most titles ran once, so calendars show that the situation exists but cannot estimate how often an organiser faces it. That share, p, has to come from organiser interviews.

Market size. We size the market bottom-up (Table 1): the number of organisers who could run multi-session sign-ups, times p, times the price P of S$120 per organiser-year (§2). Singapore has six autonomous universities and five polytechnics [3, 4]; five of the universities list at least 693 student organisations, a lower bound because counts given as "more than X" enter as X and the sixth university (SUSS) and the polytechnics are not counted (288 if only pages we could download and our own counts are used). With $p =$ **[PENDING: interviews not yet done]** % (event-weighted share of organisers' events with two or more equivalent sessions; **[PENDING: interviews not yet done]** organisers interviewed), the campus total addressable market is TAM $= 693 \times p \times$ S$120 $\approx$ S$83,200 p per year, and about S$167,000 p if the more than 700 CET training providers [5] are included. The serviceable market is SAM $= s \cdot$ TAM, with s the share of organisers not bound to a closed institutional system; the first-year obtainable market, NUS student organisations at adoption share a, is SOM $= 200 \times p \times a \times$ S$120 $\leq$ S$24,000 a year (**[PENDING: s from the organiser interviews; the team's assumed adoption share a]**). We dropped a regional (ASEAN) bound because we found no citable count. Even at $p = 1$ the counted campus market is only about S$83,200 a year (more with SUSS and the polytechnics, but of the same order), so the business case rests on the time each organiser saves (§2), not on market size.

Competitors. Table 2 summarises the vendors' documentation. Eventbrite relays a released ticket automatically with a claim window, but its inventory and waitlist are per date or ticket type; no tool we scanned ties several acceptable sessions to one queue position with one seat per person, which is what removes duplicate holds.

2 Business model and cost

Customers, pricing and value. Organisers pay; participants never pay or install anything. We propose two plans: a free tier with one open workshop at a time, and an organiser plan at S$10 per organisation per month (S$120 a year) with any number

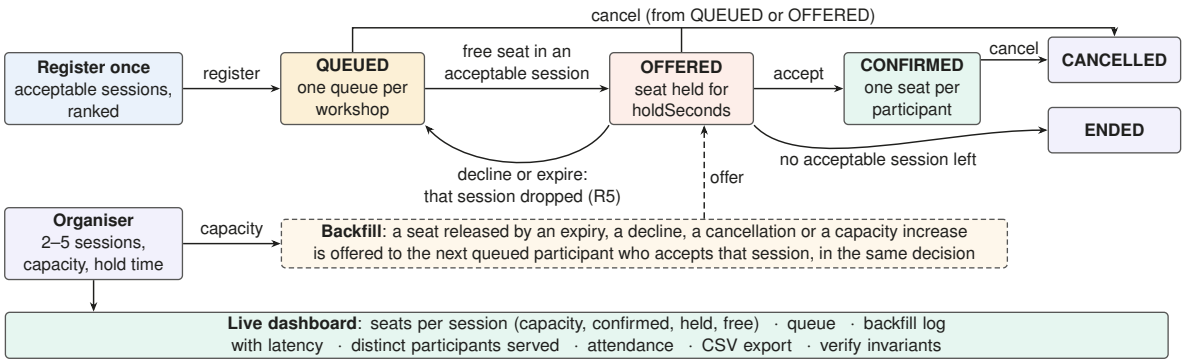


Figure 1: User flow. A participant registers once with the acceptable sessions in preference order and joins one queue per workshop. The first queued participant who accepts a session with a free seat receives an offer held for `holdSeconds`; accepting confirms the seat, declining or letting it expire drops that session from the preference list (R5). Every released seat is offered to the next queued participant in the same decision (backfill).

Table 1: Bottom-up market sizing (counts accessed 2026-10-17; own counts from the captures in `results/market/`; verification levels in `docs/market-sizing.md`).

Population	Count	Source	Estimation basis
Student organisations, five autonomous universities (NUS 200, NTU 113, SMU 150, SUTD 80, SIT 150)	≥ 693	[6–12]	official counts as lower bounds; NTU counted by us from its directory pages
CET training providers	> 700 (2016)	[5]	latest official count found; reported separately
Organisers with multi-session sign-ups	$693 \times p$	organiser interviews	p: **[PENDING: interviews not yet done]**
Price P per organiser	S$120/year	§2	S$10 per month

Table 2: Competitor scan from vendor documentation read on 2026-10-17 (not our tests). n/d: not documented in the pages we could read.

Tool	Several sessions	Waitlist	Timed relay	One seat across sessions	Organiser price
Eventbrite [13–16]	recurring event; inventory per date	per ticket type or date	yes, with claim window	no (per date)	free events free; fees on paid tickets
Luma [17–19]	workaround: cloned events	yes	no (host approves)	no	free; 5 % on paid events
SignUpGenius [20, 21]	one slot per session	per slot (paid)	relay without a hold	n/d	US$22.49/month (Gold, annual)
Peatix [22–24]	up to 50 dates	no automated waitlist	no	no	free events free; 4.9 % + S$0.99/ticket
Calendly group [25, 26]	slots with invitee limit	n/d	no	n/d	paid plans; US$10/seat/month (annual)
Google / MS Forms [27, 28]	one form per session	no	no	manual / per form	pricing not checked
SeatRelay	2–5 sessions, one queue	yes	yes, timed hold	yes (C2)	free tier; S$10/month

of open workshops and co-organisers. The prototype does not enforce plan limits; organisers already create their organisations themselves. The first channel is NUS student organisations and units that run repeated workshops, starting with the organiser of the planned pilot (§4.5). The price is defensible if $h \cdot w \cdot k \geq P$, where h is the coordination time saved per multi-session event, w the value of an organiser hour and k the number of such events per organiser-year. The organiser interviews give h (median **[PENDING: interviews not yet done]** hours per event) and the willingness to pay (median price felt to be a bargain **[PENDING: interviews not yet done]**, getting expensive **[PENDING: interviews not yet done]** S$/month). The simulation in §4.3 gives one component of h: with per-session forms, 3.87 confirmed seats per workshop end as duplicates that the organiser must chase, against 0.00 with one queue.

Cost accounting in three layers. Cloud cost is computed as usage $\times$ unit price for the 29 billable dimensions the stacks use in `ap-southeast-1`; 30 of the 31 entries of the price file (which also holds two switched-off alternatives) were read from the AWS Price List API on 2026-10-16 [29]. The layers are (1) list price, (2) after the always-free allowances [30], and (3) after credits, which is US$0 while the account's credits last. Layer 3 is why billing tools show a net cost of zero; our budget alarms therefore exclude credits, whose inclusion is the default [31]. Until measured usage is priced (**[PENDING: measured usage not yet priced (cost-snapshot.sh, then model.py --from-snapshot)]**), usage per registration is modelled from the design: 2.5 commands and 1.2 offers per registration, 6 API reads per command including polling, and 0.333 Cognito monthly active users per registration (all assumptions are listed in `results/cost/model-summary.md`).

On-premise reference. The same engine and single-page application (SPA) also run on a self-hosted stack (`services/local`: Docker Compose with PostgreSQL, Keycloak, Mailpit and nginx). That compose file is the functional reference; the priced service would run it on one mini-PC with Keycloak in production mode behind TLS, a transactional e-mail provider instead of the Mailpit sink, and backups. We have not load-tested it (**[PENDING: k6 and docker-stats run of services/local]**), so one machine's capacity (1,000,000 registrations a month) is an assumption. Two views are priced. *Increment only*: an N100 mini-PC at S$767.56 [32] with a UPS [33] and a backup disk [34], amortised over 3 years; 20.1 W at load [35] at 31.16 cents/kWh [36]; a static IP [37]; e-mail at the SES price; no labour. *Fully amortised*: plus a business fibre line at S$140 a month [38], an off-site backup copy [39] and operations labour of 4 h a month at S$60/h, which is our assumption; S$1.28 = US$1 [40]. **[PENDING: re-run the cost model with the fibre and UPS prices re-checked on 2026-10-17 (report/sources.md)]**. Load, retention and function are held equal in the model; availability is not: one machine has no failover, whereas the cloud services span several availability zones (AZs).

Result. At 1,000 registrations a month the modelled cloud list price (US$13.30) is mostly CloudWatch's fixed items, a dashboard, custom metrics and alarms (US$6.80), and Cognito monthly active users (US$5.00), which the free allowances cover; what remains is US$0.52 (Table 3). At 100,000 registrations Cognito dominates the list price (US$500.00 of US$591.24). The fully amortised on-prem view is nearly flat because labour (US$187.50) and the business line (US$109.00) dominate it. The cloud list price exceeds the increment-only view above 3,898 registrations a month and the fully amortised view above 57,278 (Table 3, right; after the free tier: 34,356 and 90,782). Below the first crossover the cloud build is the cheaper option in both views; up to the second it is cheaper than an on-prem service that pays for its own line and operations. It costs nothing out of pocket while credits last and buys multi-AZ availability, which a single volunteer-run machine gives up.

Registrations per month	Cloud: list price	Cloud: after free tier	Cloud: after credits	On-prem (3-yr): increment only	On-prem (3-yr): fully amortised
1,000	13.30	0.52	0.00	29.32	324.16
10,000	65.84	4.86	0.00	32.26	327.10
100,000	591.24	401.97	0.00	61.36	356.20
Cloud list price per 1,000 registrations: 13.30 / 6.58 / 5.91					

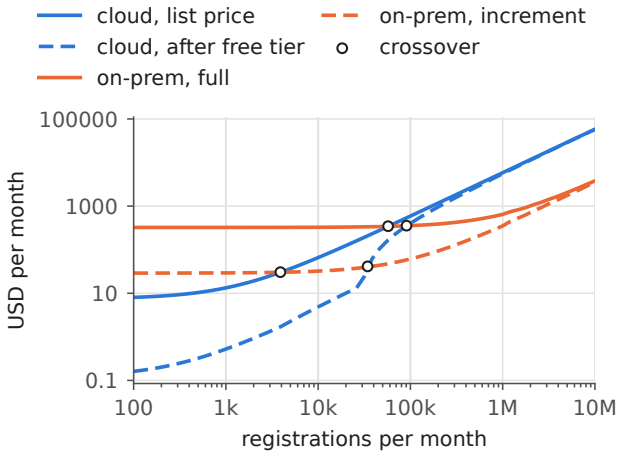


Table 3: Monthly cost in US$ by load. Left: modelled cloud usage at list price, after the free allowances and after credits, and the two on-prem views. Right: the same curves on a log–log scale with the crossover points (the after-credits layer is 0 and not drawn). Sources: `results/cost/model-summary.md`, `crossover-curve.csv`.

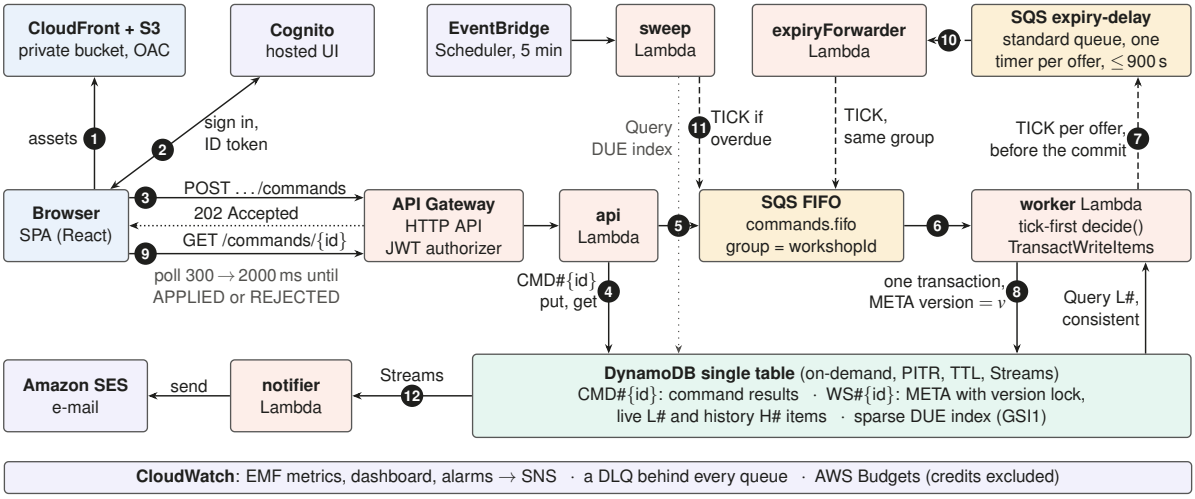


Figure 2: Cloud architecture and request path. (1) SPA from a private S3 bucket via CloudFront. (2) Sign-in on the Cognito hosted page. (3) `POST /workshops/{id}/commands` passes the JWT authorizer; (4) `api` writes `CMD#{commandId}` = ACCEPTED and (5) enqueues the command in the workshop's FIFO group, answering 202. (6) The worker, the only writer of a workshop's engine state, runs `decide()`, (7) sends one TICK per new offer to the delay queue and (8) commits one transaction guarded by the workshop version. (9) The browser polls `GET /commands/{id}`. (10) Due timers are forwarded into the FIFO group; (11) a 5-minute sweep re-ticks overdue workshops (safety net); (12) notifications go out via Streams and SES. Solid: requests; dotted: the 202 reply; dashed: queued messages; grey dotted: index query.

3 Architecture and implementation

3.1 Components and layering

SeatRelay runs in one AWS region (`ap-southeast-1`) from three CloudFormation templates: identity (Cognito user pool, Simple Email Service (SES) identity, budgets; deployed once), and application and web (private S3 bucket behind CloudFront with origin access control [41]), each deployed once per stage: `dev` for experiments, `pilot` for real users. Fig. 2 shows the application stack. Users sign in on the Cognito hosted page (authorization code flow with Proof Key for Code Exchange, PKCE), and the HTTP API validates the JSON Web Token's (JWT) signature, issuer and audience before any authenticated route runs [42]; only `/health` and the public workshop page are anonymous. Five Lambda functions (arm64, Node.js 22, traced with X-Ray) do the work: `api`, `worker`, `expiryForwarder`, `sweep` and `notifier`. Two Simple Queue Service (SQS) queues carry commands and timers: a first-in-first-out (FIFO) command queue and a standard delay queue. State lives in one DynamoDB table (on-demand capacity, point-in-time recovery (PITR), Streams, time-to-live (TTL)). We chose pay-per-request managed services because registration traffic is bursty (opening of registration, deadlines) and idle most of the time, so idle cost is close to zero (§2), and because they run across AZs without servers to patch.

All business rules are in a pure engine, `decide(snapshot, command, now)` in `packages/core`, which returns the writes, notifications, timers and backfill records of one decision and performs no I/O; thin adapters map its changes to DynamoDB items or PostgreSQL rows. The same engine runs in the cloud, in the on-prem reference and in the replay simulator (§4.3).

3.2 Write path, data model and the single-writer lock

Reads, organisation and workshop set-up, title edits and attendance are synchronous writes that do not touch the fields the engine owns. Every change to seats, queues or offers is a *command*; a user's command carries a client-generated `commandId`. The API writes the command record `CMD#{commandId}` with status ACCEPTED (conditional on absence; a repeat returns

Table 4: Rules and constraints and where they are enforced (functions in `packages/core` unless stated; full map in `docs/ARCHITECTURE.md` §9–10).

	Statement	Enforced by
R1	One live registration per account per workshop.	`register()` rejects ALREADY_REGISTERED under the single writer and version lock; mirrored by a marker item per user.
R2	Queue order = FIFO arrival order = `seq` assigned by the worker.	`seq = META.seq + 1`, written only by the workshop's single writer.
R3	Allocate in `seq` order; offer the first acceptable session, in preference order, that has a free seat.	`allocate()`, `firstAllocatableSession()`.
R4	expiresAt = min(now + holdSeconds, cutoffAt), cutoffAt = start − 30 min; no offer after cut-off.	`offerExpiry()`, `tickFirst()`; per-offer timers and the sweep (§3.4).
R5	Decline or expiry drops that session; back to QUEUED with the same `seq`, or ENDED if none is left.	`declineOffer()`, `expireOffer()`; `allocate()` in the same decision.
R6	Cancellation exits completely; the seat is relayed at once; re-registering gets a new `seq`.	`cancelRegistration()`; relay in the same transaction.
R7	Capacity below confirmed + held is rejected; an increase allocates at once; cancelling a session revokes its offers and re-queues its confirmed participants.	`setCapacity()`, `cancelSession()` and `drainCancellations()`.
C1	No over-capacity: confirmed + held ≤ capacity.	Offers only when free > 0; counters change only in `decide()` and commit atomically; `checkInvariants()`.
C2	At most one OFFERED or CONFIRMED seat per account per workshop.	R1 plus one current offer per registration; only QUEUED registrations receive offers.
C3	A command takes effect at most once.	Client `commandId`; conditional record; FIFO deduplication; record condition inside the effect transaction.
C4	**[PENDING: C4 wording from the Preliminary report]**	Candidates: per-workshop ordering (FIFO group, R2); cut-off (R4); notifications written in the state change's transaction; audit trail (command `acceptedAt`/`settledAt`, backfill records with latency, history items).
C5	Expiry is decided by server time when a command is processed; a late tick cannot invalidate a newer offer (reworded, §3.4).	`tickFirst()` before every command; ACCEPT checks `expiresAt`; ticks never expire an offer by id.
C6	**[PENDING: C6 wording from the Preliminary report]**	Same candidates as C4.
C7	Organiser actions only by members of the workshop's organisation.	Membership checks in the API (403/404) and again in the worker; command results visible only to issuer and organisers; OFFER_NOT_YOURS.

the existing record), enqueues the command on the FIFO queue with message group = workshop and deduplication id = `commandId` [43], and answers 202. If the enqueue fails, the record is deleted and the API answers 503, so no browser polls an orphan. SQS FIFO hands the messages of one group to the function one at a time and in order [44, 45]; each workshop therefore has exactly one serial writer, while different workshops are processed in parallel (worker concurrency 5).

The table keeps a workshop in one partition `WS#{id}`: the *live* items that a decision needs under prefix `L#` (workshop META item, sessions, live registrations and offers, pending backfills, a due-date marker) and settled *history* under `H#` (archived registrations and offers, backfill records, attendance). The worker reads only `L#` with a strongly consistent query, so its snapshot is bounded by capacity and queue length, not by the workshop's age. It runs `decide()`: expire what is due (§3.4), apply the command or reject it with a deterministic reason, allocate free seats in queue order, and recompute the next due time. It then commits *one* `TransactWriteItems` [46]; when the decision changes the workshop, the META item is conditioned on `version =` the snapshot's version and incremented. This optimistic lock covers the whole workshop and is a second line of defence behind the single writer; a version conflict reloads and decides again (at most three times). The command record is conditioned on being absent or still ACCEPTED in the same transaction, so a command's effects and its settlement commit together or not at all, and a redelivered or duplicated command fails the condition and is acknowledged without effect (C3). A transaction holds at most 100 actions, so a decision is capped at 12 offers, 12 expiries and 90 items; remaining work continues as an internal REALLOCATE command. A deterministic rejection settles the command as REJECTED; an infrastructure error returns the failed message and the rest of its batch for redelivery [47] after the 120 s visibility timeout, and three failed receives move it to an alarmed dead-letter queue (DLQ). Command records expire after 30 days and notifications after 90 days (TTL).

3.3 Rules R1–R7 and constraints C1–C7

Table 4 maps each rule and constraint of the Preliminary report to its enforcement. Every rule and every rejection reason has a unit test, and a property-based test [48, 49] applies random command sequences with random clock jumps and checks the invariants after every step (§4.2). Organisers can run the same invariant check on a live workshop from the dashboard (`verify`).

3.4 Responses to the TA's feedback

(1) Market size. Added in §1 (Table 1).

(2) Per-offer timers instead of sweep-bounded expiry. In the Preliminary design a periodic sweep expired offers, so the sweep interval bounded the backfill latency. Each offer now has its own timer (Fig. 3b). SQS FIFO queues do not support per-message timers, and a message timer is at most 15 minutes [50]. The worker therefore sends one TICK per new offer to a *standard* delay queue with DelaySeconds = min(900, ⌈expiresAt − now⌉), *before* its commit (a timer whose commit fails finds nothing to expire). When the TICK becomes visible, the `expiryForwarder` moves it into the workshop's FIFO group (deduplication id `tick#offerId#expiresAt`), so expiries take the same ordered single-writer path as user commands; a TICK that is still early (holds over 15 minutes) returns to the delay queue. A TICK is only a wake-up call: every decision begins by

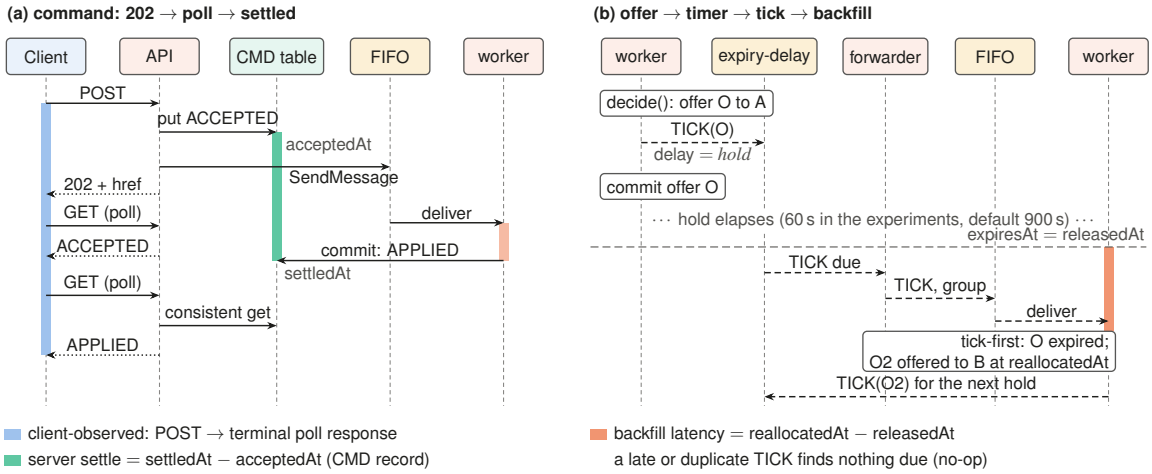


Figure 3: Request sequences (not to scale). (a) A command is accepted (202) once its record is written and it is enqueued; the browser polls until the worker's transaction has settled it. Server settle latency = `settledAt` − `acceptedAt`; the client-observed latency adds the POST and GET round trips and up to one poll interval. (b) Each offer schedules its own timer; at `expiresAt` the forwarder moves the TICK into the workshop's FIFO group, and the worker expires the offer (tick-first) and relays the seat in the same decision. Backfill latency = `reallocatedAt` − `releasedAt`, with `releasedAt` = `expiresAt` for an expiry.

expiring all offers with expiresAt $\leq$ now (*tick-first*). Hence late or duplicate ticks are no-ops, an ACCEPT processed after `expiresAt` is rejected whether or not its timer has arrived, and a late tick cannot touch a newer offer; this is the reworded C5. The 5-minute sweep stays as a safety net: each commit maintains the workshop's next due time on a sparse index, and the sweep enqueues a TICK for every overdue workshop (lost timers, session cut-offs, lost continuations); `SweepLateExpiries` counts offers a sweep tick expired more than 60 s after `expiresAt` (expected 0). The sweep interval therefore no longer bounds the backfill latency, whose budget for an expiry is the delay-queue rounding (whole seconds), one forwarder invocation, one FIFO enqueue and one worker decision; a decline or cancellation relays the seat inside the releasing transaction (Table 5, Backfill row). One-time EventBridge Scheduler schedules [51] would lift the 15-minute limit but add a service, a schedule per offer and a role.

(3) How the browser learns that a 202 has settled: polling. The 202 response carries `href` and `pollAfterMs` = 300 (Fig. 3a). The browser polls `GET /commands/{id}`, a strongly consistent read, after 300, 500 and 1000 ms and then every 2000 ms; after 30 s it polls every 5 s and shows a "still processing" note. The terminal state is APPLIED with a result or REJECTED with a reason; the result contains the participant's registration summary (status, queue position, offer expiry), so the interface updates from the poll alone and separates "request received" from "seat confirmed". A retry reuses the `commandId`. Changes caused by other people, such as an offer created by a backfill, reach the status page through a 10 s refresh while a registration is QUEUED or OFFERED and the tab is visible. We did not use WebSockets: at pilot scale they would add an API type, a connection table and a failure path for little gain, and polling is bounded (about 6 reads per command in the cost model). We report the server settle latency (`settledAt` − `acceptedAt`, both on the command record; the worker takes `settledAt` when it has decided, just before its commit) separately from the client-observed latency, which adds the POST and GET round trips and up to one poll interval.

3.5 Availability, security and privacy

Within the region every managed service is multi-AZ, so an AZ failure needs no action. A worker crash delays only its own workshop until the message is redelivered (120 s visibility timeout); other workshops continue. A region failure is not covered: the table could be restored from PITR, whose latest restorable time is typically five minutes in the past [52] (a recovery point objective of about five minutes for workshop data), but user identities live in one regional Cognito user pool without backup and every user id in the table is a Cognito subject, so accounts would have to be recreated and ids remapped. This path is untested; we claim no recovery time objective. The actor of every command comes only from the verified token, all input is schema-validated, and organiser rights are checked in the API and again in the worker (C7). Each function has its own role on this stack's resources: the sweep only reads the table and the forwarder cannot access it, while `api` and `worker` use SAM's table-level read-write template, which could be narrowed. The API is throttled per stage and admits cross-origin requests only from the stage's CloudFront origin and the local development origin; the experiment scripts' password sign-in client is also an accepted audience, and both stages share one user pool. The S3 bucket is private. The application stores e-mail address, display name and Cognito user id; API access logs (source IP, user agent) and function logs are kept for 14 days; raw request events are never logged. E-mail goes through SES, which in its sandbox delivers only to verified addresses [53]; until production access is granted, pilot participants rely on in-app notifications.

4 Evaluation

4.1 Method

Commands went through the `dev` stage's public API as the browser sends them: client `commandId`, POST, 202, then polling with the back-off of §3.4. Crashes were injected with `_fault` flags that only `dev` honours, stale TICKs were sent directly to the FIFO

Table 5: Acceptance targets of the Preliminary report vs measured values (dev stage, 3 runs per experiment unless stated). Source directories: `results/2026-10-17-<experiment>[-<n>]/`, `results/loadtest/`, `results/replay/`, `results/cost/`.

Experiment	Target	Measured	Verdict
Core correctness	0 over-capacity, 0 double holds, every step as expected, `verify` ok	9 scenarios × 3 runs: 672/672 checks; over-capacity 0, double holds 0; 234 verify calls, 0 failed	PASS
Preference, order	Never offer an unacceptable session; preference order respected (engine property test and 20 cases on dev)	Engine: **[PENDING: core property-test run not yet recorded]**. Dev: 60/60 cases (20 × 3 runs), 0 unacceptable offers	PASS (dev part)
Concurrency	Invariants hold after each concurrent phase (50 accounts; one account's concurrent ACCEPT, DECLINE, CANCEL); version-conflict retries recorded (expected 0)	50 accounts race for 5 seats in 3 sessions: `verify` found no violation after any phase; retries 0 (inconclusive, see text); 76 of 209 requests got HTTP 503 (see text); same-account races 5/15 as expected, each of the 10 others with an HTTP 503; 22/39 checks	FAIL
Repeat execution	Same `commandId` sent repeatedly and crash after commit: applied once, one record	12/12 probes with 5 concurrent copies each; 6/6 crash-after-commit probes, redelivery acknowledged without a second effect	PASS
Expiry race	ACCEPT at `expiresAt`−200 ms applied, at +200 ms rejected OFFER_EXPIRED whatever the timer did; a stale tick has no effect	Early applied 3/3; late rejected 3/3 (timer first in 3, see text); stale ticks without effect 3/3	PASS
Failure recovery	Crashes before commit, at the timer-sending step and after commit recover, settle once and are traceable; notifier backlog delivered < 300 s after an outage	Before commit 3/3 and timer step 3/3 settled on the redelivery (median 120.1 and 120.1 s); after commit 3/3 settled at once (median 0.1 s), redelivery acknowledged; dead letters 0; backlog of 10 notifications processed ≤ 52 s after restore, all SKIPPED (no e-mail on dev)	PASS (e-mail not sent)
Access control	Unauthorised requests refused (401/403/404, OFFER_NOT_YOURS); nothing changed	165/165 probes in 7 groups as expected	PASS
Performance	Read and 202 p95 < 1 s; settle p95 < 5 s (server and client); errors < 1 %†	Targets met in 2 of 6 levels (10 VUs). 25 VUs, cross-workshop: settle 6,302 (server) / 6,912 (client) ms, errors 1.57 %. 50 VUs: read 284, 202 300, settle 28,460 / 29,127 ms, errors 9.81 % (hot spot: settle 29,335 ms, errors 8.01 %); Fig. 4	FAIL
Backfill latency	Expiry-driven p95 ≤ 120 s; every expiry relayed; `SweepLateExpiries` = 0	Run aborted: HTTP 503 when creating its workshop (see text); **[PENDING: backfill re-run]**. Interim, from the smoke and failure-recovery runs: 4 timer-path expiry relays, max 982 ms; cancellation relay 0 ms	FAIL
Cost	Unit cost and trend by scale in three layers; on-prem comparison	Model: US$13.30 → US$5.91 per 1,000 registrations at list price (1k → 100k a month); crossovers in §2. Measured usage: **[PENDING: measured usage not yet priced (cost-snapshot.sh, then model.py --from-snapshot)]**. On-prem reference: not load-tested (§2)	model
Mechanism comparison	Duplicate holds, distinct participants served and refill time, UNIFIED vs PER_SESSION, 95 % CIs	1,000 simulated workshops: duplicate-hold events 0.00 vs 34.57; distinct served, per-session − unified: −4.04 [−4.25, −3.84]; §4.3	done (simulation)
User study	Completion ≥ 90 %; paired task time −25 %; System Usability Scale (SUS) ≥ 75; 12 participants	**[PENDING: user study not yet run]**	–
Real pilot	Two real sessions run with SeatRelay	**[PENDING: pilot not yet run]**	–

queue, and the notifier outage was simulated by setting its reserved concurrency to 0. On `dev` e-mail is disabled, so the notifier records each notification as SKIPPED without sending it. Each script asserts its checks, calls `verify` after its scenario steps and writes `results/<date>-<experiment>/`; each experiment ran 3 times in one invocation, and the latest complete run is quoted. Expiry experiments use holdSeconds = 60 so that a sample does not take 15 minutes. The acceptance smoke test passed before the series (13/13 checks, run `c527cac7`). The 8 quoted runs recorded 4 commits (`227de3d, ec54916, c1d0660, 00faa52`); 7 runs had uncommitted changes in the working tree: **[PENDING: confirm that the dev Lambda code was not redeployed between these commits and that only results/ and docs/ were uncommitted, or re-run from a clean checkout]**. Run ids and commits of every number are listed in `report/generated/numbers.tex`.

4.2 System results against the acceptance targets

Table 5 has one row per acceptance target of the Preliminary report (its Table 3, as restated in our execution plan; † marks a threshold we added).

Failed checks. In their latest runs 2 of the 8 experiments failed a check (concurrency, backfill), and we report them as failures. In the *concurrency* run, 76 of 209 requests received HTTP 503 responses without an application error code, so they did not come from the API's own 503 path (failed enqueue). Because some of these commands were nevertheless applied, the harness's expected sequence numbers and confirmation counts no longer matched, although `verify` found no invariant violation after any phase. The version-conflict metric read 0, but CloudWatch returned no samples for any SeatRelay metric in that window, so that reading is inconclusive. The *backfill* run aborted at its first request, creating a workshop, with the same kind of 503, so it produced no latency samples. The pilot stack was deployed to the same account around the time of the concurrency run (its deployment record falls inside the run window), a possible confounder for the re-run to exclude. **[PENDING: root cause of the HTTP 503 responses, then clean concurrency and backfill re-runs]**. An earlier *expiry-race* run failed one check because a probe meant to reach the API 200 ms before `expiresAt` arrived after it (client send-timing jitter)

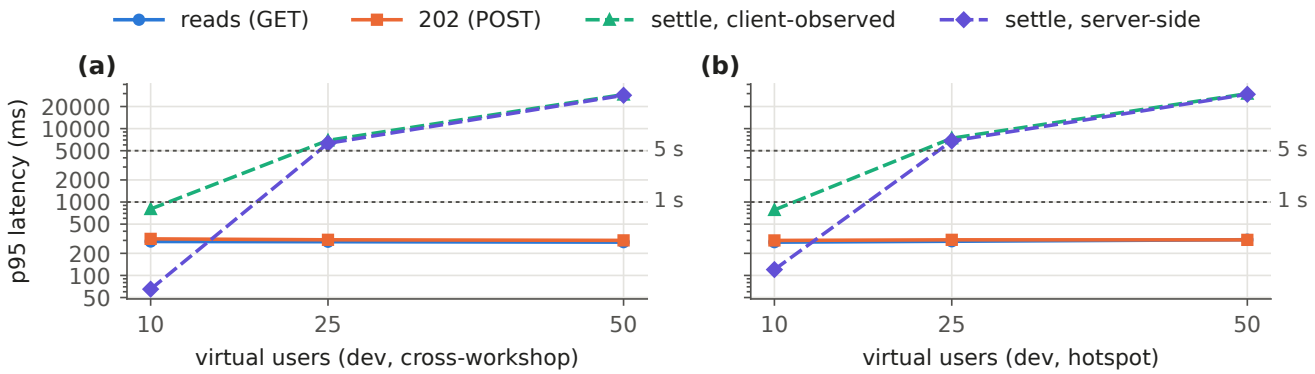


Figure 4: Load test on dev: p95 latency of reads, 202 responses and settlement (server and client) by number of virtual users, (a) spread over 10 workshops, (b) one hot workshop; median of 3 rounds per level, dotted lines at the 1 s and 5 s targets. Source: `results/loadtest/2026-10-17/raw.csv`.

[PENDING: backfill-latency chart (results/<date>-backfill)]

Figure 5: Expiry-driven backfill latency (`reallocatedAt` − `expiresAt`) on dev with holdSeconds = 60, histogram and empirical cumulative distribution against the 120 s target. Source: `results/<date>-backfill/raw.csv`.

and was rejected as expired, which agrees with the server clock; the harness now calibrates the send time, and the quoted run (`e1be2b0b`) is a re-run. In all 3 late trials the timer's tick had expired the offer before the ACCEPT arrived, so the opposite order (ACCEPT before the timer) was not observed on `dev`; the engine's unit tests cover it. No stale tick changed a newer offer.

Latency. In the correctness run the server settled commands with p95 59 ms (207 commands); the client observed p95 837 ms, because it adds the first 300 ms poll interval, the request round trips and up to one further interval. In the concurrency run, where all commands of one workshop are serialised in one FIFO group by design, the server p95 rose to 6,349 ms, above the 5 s settle target (the same run had the 503 responses above). After a worker crash a command settles when the 120 s visibility timeout returns the message (median 120.1 s), which bounds the recovery of a stalled workshop. In the k6 load test [54] with 10–50 virtual users (VUs) (Fig. 4), reads and 202 responses stayed fast, but settlement did not keep up above 10 VUs, in the cross-workshop scenario as in the hot spot: the commands queue backed up (oldest message 17 s at 50 VUs over 10 workshops), so per-workshop serialisation alone does not explain it. **[PENDING: cause of the settle backlog and HTTP errors at 25–50 VUs]**.

4.3 Mechanism comparison: one queue vs per-session forms

The replay tool runs the production engine with a simulated clock and ideal timers on 1,000 random workshops (2–5 sessions, 1.2–3 times oversubscribed, holds of 60–900 s, random accept, decline, ignore, cancel and hoarding behaviour). Each seed's identical arrival stream is replayed twice: in UNIFIED mode (SeatRelay) and in PER_SESSION mode, where a participant registers separately for every acceptable session, each session has its own queue, and duplicates are released by hand only with some probability. With one queue there were no duplicate holds, as C2 requires; with per-session queues there were 34.57 duplicate-hold events per workshop [33.50, 35.61] (Fig. 6). One queue served 58.08 distinct participants per workshop against 54.03 (paired difference −4.04 [−4.25, −3.84]; fewer under PER_SESSION in 89.9 % of seeds, equal in 9.1 %), and a released seat reached a new participant after 4.30 instead of 15.72 minutes on average (847 seeds with refills in both modes). Seats left empty at cut-off did not differ (difference 0.09 [−0.01, 0.19]). The comparison is a simulation with an assumed behaviour model, not a field measurement; the plan's additional replay on the deployed system was not carried out.

4.4 User study

The protocol has twelve people outside the team (at least four with organising experience) use SeatRelay and a control that mirrors current practice, one Google Form per session plus a shared Sheet with duplicate-detection formulas, in counterbalanced order with equivalent data sets. Participants perform five tasks (choose acceptable sessions, join the waitlist, interpret their status, confirm, cancel) and organisers four (create a multi-session workshop, handle a cancellation, inspect the backfill, read the distinct count and export). We measure completion, paired task time (Wilcoxon signed-rank test [55]), SUS [56], erroneous actions, help requests and the organiser's manual steps. **[PENDING: user-study results: completion, paired time with Wilcoxon p, SUS with 95 % CI, errors and help requests per condition, organiser manual steps]**

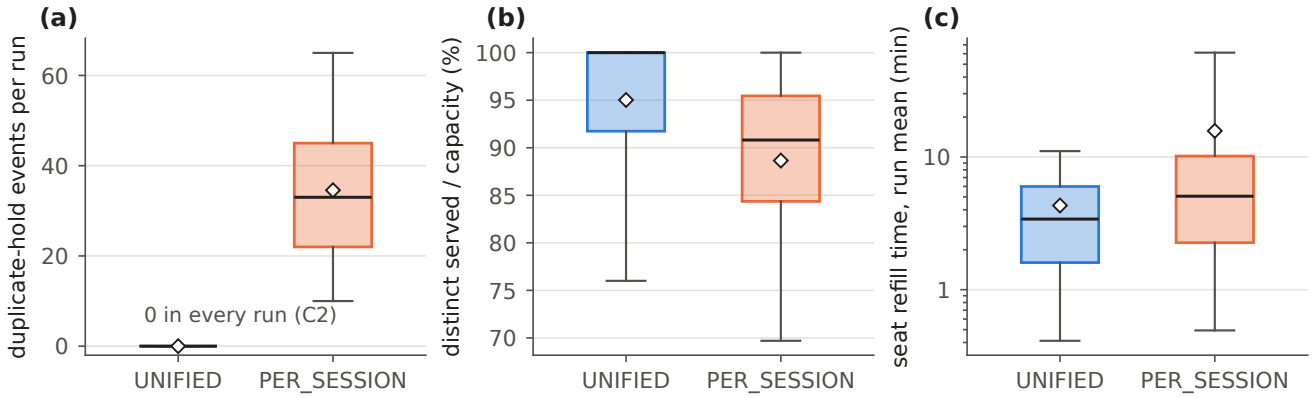


Figure 6: Mechanism comparison over 1,000 simulated workshops (one box per mode: p25–p75, median line, whiskers p5–p95, diamond = mean). (a) Duplicate-hold events per workshop; (b) distinct participants served as a share of capacity; (c) mean seat refill time per workshop (log scale). Source: `results/replay/raw.csv`.

[PENDING: user-study charts: paired task time and SUS (results/user-study/)]

Figure 7: User study: total task time per participant in both conditions (left) and SUS scores (right), SeatRelay vs one form per session plus a Sheet.

4.5 Real pilot

The `pilot` stage (fault injection disabled, e-mail enabled; while SES is in the sandbox only verified addresses receive mail and others rely on in-app notifications) is deployed to serve a real workshop in two sessions with registration open for at least five days; the organiser's CSV exports and the dashboard's backfill log will be the evidence. **[PENDING: pilot workshop, dates and observations: natural cancellations, backfills, notifications, attendance, organiser feedback]**.

5 Conclusion and future work

SeatRelay replaces one form per session with one registration, one queue and a timed hold per workshop, and relays every released seat automatically. The cloud design serialises each workshop through one FIFO message group and commits every decision in one conditional DynamoDB transaction. On the deployed system 6 of the 8 system experiments met their targets (preference order on its dev part; failure recovery without e-mail delivery, which is off on `dev`), no `verify` call found an invariant violation, and the load test met its targets only up to 10 virtual users. By design, per-offer timers with tick-first semantics take the sweep interval off the backfill path; the 4 timer-path relays observed so far took at most 982 ms. The 202 protocol is settled by bounded polling. In simulation, one queue removed duplicate holds and served more distinct participants than per-session queues. In our usage model the cloud build costs US$13.30 a month at list price for 1,000 registrations and is cheaper than a self-hosted machine that pays for its line and operations up to 57,278 registrations a month. Hypotheses of the Preliminary report: **[PENDING: H1–H3 wording and verdicts; evidence: queue position in the user study (H1), backfill latency (H3)]**.

Open items are the HTTP 503 responses that failed the concurrency run and aborted the backfill run, the settle backlog under load, the backfill measurement, the on-prem load test, the interviews that fix p and the price, the user study and the pilot. Future work: (i) batch several commands of one workshop per worker invocation with a shared snapshot to raise throughput; (ii) replace polling with server push only if the user study shows that the 10 s status refresh is too slow; (iii) enforce the plan limits and add billing for the organiser plan; (iv) a second region with table replication and a replicated or exported identity store if an availability target beyond one region is required; (v) the Cognito Lite tier, since Cognito dominates the cost at high volume.

6 Code explanation

The code, infrastructure templates, experiment scripts, raw results and this report's sources are public at https://github.com/gengyuzccc-cell/Team18_CS5224_SeatRelay_Project_AY2627Sem1 (`README.md` is the long form of this section). It is a pnpm workspace in TypeScript (Node.js 22): `packages/contracts` (zod schemas shared by all layers), `packages/core` (the pure engine), `packages/service` (API routes, worker protocol, sweep), `services/aws` (SAM templates, five Lambda

handlers, DynamoDB/SQS/SES adapters), `services/local` (the on-prem reference: Fastify, PostgreSQL, Keycloak, Mailpit, nginx), `apps/web` (React SPA), `tools/experiments`, `tools/loadtest` (k6), `tools/replay`, `tools/market` (page captures and counts), `results/` (raw outputs; the only source of measured numbers) and `report/`.

```
corepack enable && pnpm install --frozen-lockfile
pnpm lint && pnpm typecheck && pnpm test                          # all packages, as in CI
STAGE=dev ALARM_EMAIL=<e-mail> tools/scripts/bin/deploy-all.sh  # bundles, sam deploy, SPA upload
pnpm local:up                                                     # on-prem reference (Docker Compose)
TARGET=dev tools/experiments/bin/run-table3.sh                    # acceptance-target experiments, then k6
pnpm replay --seeds 1000                                          # mechanism comparison -> results/replay/
python3 tools/experiments/cost/model.py                           # cost model -> results/cost/
python3 -I tools/market/count.py                                  # market own counts -> results/market/
python3 -I report/tools/report_numbers.py && python3 -I report/tools/make_figures.py   # numbers, charts
./report/build.sh -t Final -m 10                                  # this PDF, with its checks
```

Table 6: Where the main claims are implemented (paths relative to the repository root).

Claim	Implementation
Tick-first decision, R1–R7, invariants	`packages/core/src/{decide,tick,apply,allocate,drain,invariants}.ts`
Idempotent intake (C3); transaction with version lock	`packages/service/src/api/routes/commands.ts`; `services/aws/src/adapters/dynamo/store.ts`
Worker protocol, retries, fault injection	`packages/service/src/worker/processCommand.ts`; `services/aws/src/handlers/worker.ts`
Per-offer timers and forwarder	`services/aws/src/adapters/sqs.ts`; `services/aws/src/handlers/expiryForwarder.ts`
Sweep safety net	`packages/service/src/worker/sweep.ts`; `services/aws/templates/app.yaml`
Polling back-off (browser)	`apps/web/src/commands/{backoff,pollCommand,commandRunner}.ts`
Access control (C7)	`packages/service/src/api/context.ts`
Experiments, replay, cost, report numbers	`tools/experiments/src/*.ts`; `tools/replay`; `tools/experiments/cost/model.py`; `report/tools/report_numbers.py`

Assumptions

- *Market:* organisation counts are official lower bounds; NUS, SMU and SIT figures were read from search excerpts of the official pages because direct download was blocked (**[PENDING: dated browser screenshots of these three pages]**); p, s and the value per event come only from the organiser interviews.
- *Cost:* usage per registration is modelled (§2) until measured usage is priced; X-Ray is priced at 100 % sampling (an upper bound); operations labour and the on-prem machine's capacity are assumptions (the on-prem stack has not been load-tested); availability is not equalised between the two builds.
- *Experiments:* the `dev` stage, a 60 s hold for expiry experiments and synthetic accounts; the mechanism comparison uses an assumed behaviour model and ideal timers.
- *Operations:* one AWS region; until SES production access is granted, e-mail reaches only verified addresses and participants rely on in-app notifications.

AI usage declaration

Where. The application code, infrastructure-as-code templates, tests, experiment and load-test scripts, analysis and cost-model scripts, the market page captures and counts, the competitor scan, the documentation, and the drafts of this report and the slides were generated with Claude Code, an AI coding assistant. *How.* The team set the scope, rules, constraints and acceptance targets in the Preliminary report and the execution plan, directed the work step by step, and reviewed the generated code, documents and figures. The team created and owns the AWS account and the repository. The deployment, experiment, load-test, replay and cost-model scripts were run by the assistant in a development session, at the team's request and with the team's AWS credentials; the assistant also did the desk research for §1. The team is responsible for the interviews, the user study and the pilot. **[PENDING: name of the team member who checked every reported number against results/]**. The assistant did not type or alter any measured value: every experimental, simulation and own-count number in this report is produced by a script from a raw file in `results/`, external figures are cited, and data that does not exist yet is marked as pending. *What we learned.* **[PENDING: member learning paragraphs: one "what I learned" paragraph of at most 50 words per team member, written by that member]**

7 References

All web pages were accessed on 2026-10-17 unless stated.

[1] NUS Libraries. *NUS Libraries events calendar (LibCal)*. https://nus.libcal.com/calendar.

[2] SMU Libraries. *SMU Libraries events calendar (LibCal)*. https://smu-sg.libcal.com/calendar.

[3] Ministry of Education, Singapore. *Autonomous universities*. Mar. 5, 2026. https://www.moe.gov.sg/post-secondary/overview/autonomous-universities.

[4] Ministry of Education, Singapore. *Overview of post-secondary education institutions (PSEI)*. Jan. 23, 2026. https://www.moe.gov.sg/post-secondary/overview.

[5] SkillsFuture Singapore. *Steady progress in implementation of SkillsFuture Credit*. https://www.skillsfuture.gov.sg/newsroom/steady-progress-in-implementation-of-skillsfuture-credit.

[6] NUS Office of Student Affairs. *Get started (student leader resources)*. https://osa.nus.edu.sg/experience-communities/leadership/student-leader-resources/get-started/.

[7] Nanyang Technological University. *Special interest clubs*. https://www.ntu.edu.sg/life-at-ntu/student-life/student-activities-and-engagement/clubs-groups-societies/special-interest-clubs.

[8] Nanyang Technological University. *Students' Union Council*. https://www.ntu.edu.sg/life-at-ntu/student-life/student-activities-and-engagement/clubs-groups-societies/ntu-students-union-council.

[9] Nanyang Technological University. *Clubs & societies*. https://www.ntu.edu.sg/life-at-ntu/student-life/student-activities-and-engagement/clubs-groups-societies.

[10] Singapore Management University. *Office of Student Life (OSL) – Freshmen Orientation 2022*. https://orientation.smu.edu.sg/2022/student-life/offices/office-of-student-life-osl.

[11] Singapore University of Technology and Design. *Student organisations and Fifth Row*. Sept. 25, 2026. https://www.sutd.edu.sg/campus-life/student-life/student-organisations-fifth-row/.

[12] Singapore Institute of Technology. *Find your tribe*. https://www.singaporetech.edu.sg/life-at-sit/find-your-tribe.

[13] Eventbrite. *Create multiple date or recurring events*. https://www.eventbrite.com/help/en-us/articles/256014/create-multiple-date-or-recurring-events/.

[14] Eventbrite. *How to set up an event waitlist*. https://www.eventbrite.com/help/en-us/articles/811539/how-to-set-up-an-event-waitlist/.

[15] Eventbrite. *Event waitlist management for sold-out events*. https://www.eventbrite.com/features/waitlist/.

[16] Eventbrite. *Eventbrite pricing and features for organisers (Singapore)*. https://www.eventbrite.sg/organizer/pricing/.

[17] Luma. *Multi-session / recurring events*. https://help.luma.com/p/multi-session-recurring-events.

[18] Luma. *Waitlist*. https://help.luma.com/p/waitlist.

[19] Luma. *Pricing*. https://luma.com/pricing.

[20] SignUpGenius. *How to use the SignUpGenius waitlist feature*. https://www.signupgenius.com/blog/waitlist-feature.

[21] SignUpGenius. *Pricing*. https://www.signupgenius.com/pricing.

[22] Peatix. *How to create recurring events*. https://help-organizer.peatix.com/en/support/solutions/articles/44002645658-about-recurring-events.

[23] Peatix. *Can I create a waiting list?* https://help-organizer.peatix.com/en/support/solutions/articles/44002027169-can-i-create-a-waiting-list-.

[24] Peatix. *Pricing and features (Singapore)*. https://services.peatix.com/en/sg.

[25] Calendly. *Group event type overview*. Oct. 13, 2026. https://calendly.com/help/group-event-type-overview.

[26] Calendly. *Pricing*. https://calendly.com/pricing.

[27] Google. *View & manage form responses*. https://support.google.com/docs/answer/139706?hl=en.

[28] Microsoft. *Adjust your form or quiz settings in Microsoft Forms*. https://support.microsoft.com/en-us/forms/adjust-your-form-or-quiz-settings-in-microsoft-forms.

[29] AWS. *Getting price list files using the AWS Price List Bulk API*. https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/using-the-aws-price-list-bulk-api.html.

[30] AWS. *AWS Free Tier*. https://aws.amazon.com/free/.

[31] AWS. *CostTypes*. https://docs.aws.amazon.com/aws-cost-management/latest/APIReference/API_budgets_CostTypes.html.

[32] Newegg Global (Singapore). *Beelink Mini S12 Pro mini PC, Intel N100, 16 GB DDR4, 500 GB SSD*. price read from a search-engine excerpt on 2026-10-16. https://www.newegg.com/global/sg-en/p/3C6-01D0-000R1.

[33] SourceIT. *APC Back-UPS CS 650VA 230V ASEAN (BK650-AS)*. https://sourceit.com.sg/products/apc-back-ups-cs-650va-230v-asean-bk650-as.

[34] element14 Singapore. *WD Elements Portable 2 TB (WDBU6Y0020BBK-EESN)*. price read from a search-engine excerpt on 2026-10-16. https://sg.element14.com/wd/wdbu6y0020bbk-eesn/disk-drive-2-5-2tb-usb-3-0/dp/3532741.

[35] Notebookcheck. *Intel N100 performance debut: Beelink Mini S12 Pro mini PC review*. https://www.notebookcheck.net/Intel-N100-performance-debut-Beelink-Mini-S12-Pro-mini-PC-review.758950.0.html.

[36] SP Group. *Tariff information (electricity tariff, 1 Oct – 31 Dec 2026)*. https://www.spgroup.com.sg/our-services/utilities/tariff-information.

[37] HardwareZone Forums. *Singapore ISP comparisons (static IP options)*. https://forums.hardwarezone.com.sg/threads/singapore-isp-comparisons.6665380.

[38] M1. *Business fibre broadband plans*. https://www.m1.com.sg/business/deals/fibre-broadband.

[39] Backblaze. *B2 cloud storage pricing*. https://www.backblaze.com/cloud-storage/pricing.

[40] MoneyExchangeRate.org. *USD to SGD exchange rate*. https://moneyexchangerate.org/currencyexchange/usd/sgd/262.

[41] AWS. *Restrict access to an Amazon S3 origin*. https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html.

[42] AWS. *Control access to HTTP APIs with JWT authorizers in API Gateway*. https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html.

[43] AWS. *Exactly-once processing in Amazon SQS*. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues-exactly-once-processing.html.

[44] AWS. *Amazon SQS FIFO queue key terms*. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-key-terms.html.

[45] AWS. *Configuring scaling behavior for SQS event source mappings*. https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-scaling.html.

[46] AWS. *TransactWriteItems*. https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_TransactWriteItems.html.

[47] AWS. *Handling errors for an SQS event source in Lambda*. https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-errorhandling.html.

[48] K. Claessen et al. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs". In: *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. ACM, 2000, pp. 268–279. DOI: 10.1145/351240.351266.

[49] N. Dubien. *fast-check: Property based testing for JavaScript and TypeScript*. Version 3.23.2. https://fast-check.dev/.

[50] AWS. *Amazon SQS message timers*. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-message-timers.html.

[51] AWS. *Schedule types in EventBridge Scheduler*. https://docs.aws.amazon.com/scheduler/latest/UserGuide/schedule-types.html.

[52] AWS. *Point-in-time recovery: how it works*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery_Howitworks.html.

[53] AWS. *Request production access (moving out of the Amazon SES sandbox)*. https://docs.aws.amazon.com/ses/latest/dg/request-production-access.html.

[54] Grafana Labs. *Grafana k6 documentation*. https://grafana.com/docs/k6/latest/.

[55] F. Wilcoxon. "Individual Comparisons by Ranking Methods". In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.

[56] J. Brooke. "SUS: A "Quick and Dirty" Usability Scale". In: *Usability Evaluation in Industry*. Ed. by P. W. Jordan et al. London: Taylor & Francis, 1996, pp. 189–194.